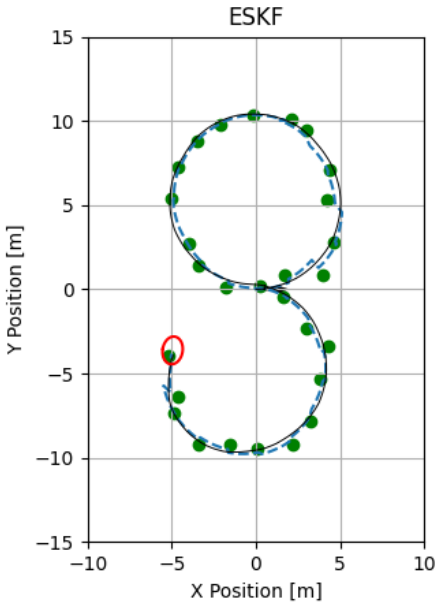
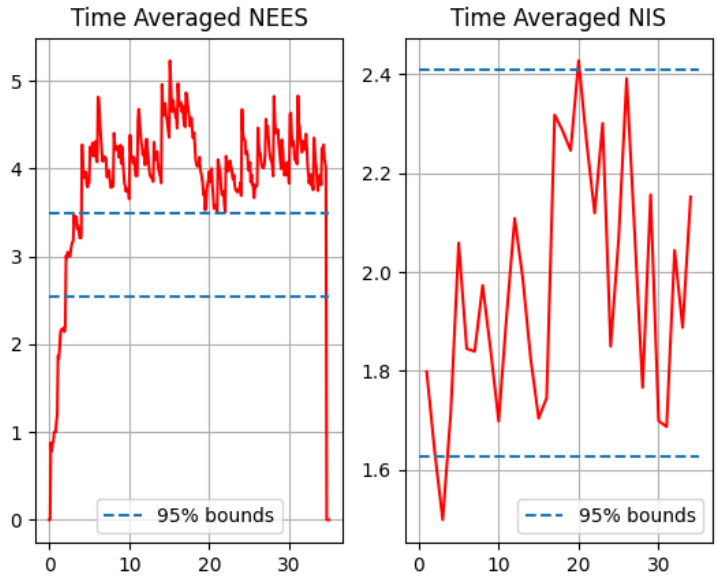
3.e)

Figure 1: Animation of ESKF



3.f)

Figure 2: Monte Carlo Evaluation of ESKF



Code Snippet 1: Kalman Filter Code

```python
import numpy as np
from Code.Filtering.filter_performance_metrics import NIS

def discrete_covariance_prediction(P_k_k, F_k, Q_k):
    P_kp1_k = F_k @ P_k_k @ F_k.T + Q_k
    return P_kp1_k

def Kalman_gain(P_k_km1, H, R_z):
    S_k = H @ P_k_km1 @ H.T + R_z
    S_k_inv = np.linalg.inv(S_k)
    K_k = P_k_km1 @ (H.T @ S_k_inv)
    return K_k, S_k_inv

def covariance_correction(P_k_km1, K_k, H, R_z):
    P_k_k = (np.eye(3) - K_k @ H) @ P_k_km1 @ (np.eye(3) - K_k @ H).T + K_k @ R_z @ K_k.T
    return P_k_k

def EKF(x_km1_km1, P_km1_km1, xi_k, h, z_k, f_disc, F, Q_k, H, R_z):
    n_x = x_km1_km1.size

    # Predict
    x_k_km1 = f_disc(x_km1_km1, xi_k)
    P_k_km1 = discrete_covariance_prediction(P_km1_km1, F(x_km1_km1, xi_k), Q_k(x_km1_km1))

    H_k = H(x_k_km1)

    # Correct if Measurement Exists
    if not np.all(np.isnan(z_k)):
        # Correct
        K_k, S_k_inv = Kalman_gain(P_k_km1, H_k, R_z)
        zhat_k = h(x_k_km1).reshape(z_k.shape) # Expected measurement
        innovation = z_k - zhat_k

        x_k_k = x_k_km1.reshape(n_x,) + K_k @ innovation
        P_k_k = covariance_correction(P_k_km1, K_k, H_k, R_z)

        # Filter performance metric (Normalized Innovation Squared - NIS)
        nu_k = NIS(innovation, S_k_inv)
        return x_k_k.reshape((n_x,)), P_k_k.reshape((n_x, n_x)), nu_k, K_k, innovation
    else:
        return x_k_km1.reshape((n_x,)), P_k_km1.reshape((n_x, n_x))

# def ESKF_update():
def ESKF(x_km1_km1, P_km1_km1, xi_k, h, z_k, f_disc, F, Q_k, H, R_z, x_update_func):
    n_x = np.size(x_km1_km1, axis = 0)

    # Predict
    x_k_km1 = f_disc(x_km1_km1, xi_k)
    P_k_km1 = discrete_covariance_prediction(P_km1_km1, F(x_km1_km1, xi_k), Q_k(x_km1_km1))

    H_k = H(x_k_km1)

    # Correct if Measurement Exists
    if not np.all(np.isnan(z_k)):
        # Correct
        K_k, S_k_inv = Kalman_gain(P_k_km1, H_k, R_z)
        zhat_k = h(x_k_km1).reshape(z_k.shape) # Expected measurement
        innovation = z_k - zhat_k

        delta_x = K_k @ innovation
        x_k_k = x_update_func(x_k_km1.reshape(x_km1_km1.shape), delta_x)
        P_k_k = covariance_correction(P_k_km1, K_k, H_k, R_z)

        # Filter performance metric (Normalized Innovation Squared - NIS)
        nu_k = NIS(innovation, S_k_inv)
        return x_k_k.reshape(x_km1_km1.shape), P_k_k.reshape((n_x, n_x)), np.squeeze(nu_k),
    K_k, innovation
    else:
```

```
        return np.squeeze(x_k_km1.reshape(x_km1_km1.shape)), P_k_km1.reshape((n_x, n_x))

# def LIEKF_update():
```

Code Snippet 2: ESKF Code

```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from Code.SO2.SO2_maps import so2_exp
from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse, se2_mat_to_tuplevec, se2_tuplevec_to_mat
from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
from Code.Plotting.covariance_plotting import plot_cov_ellipse, calc_and_plot_cov_ellipse, cov_ellipse_3D, create_cov_ellipse, plot_cov_ellipse_ax
from Code.Plotting.SE2_plotting import plot_scatter_unicycles, plot_scatter_unicycles_with_cov, plot_Lie_algebra_cov_ellipse, plot_scatter_unicycles_ax
from HW4.Parts.AAE590LGM_HW4_Problem import AAE590LGM_HW4_Problem
from Code.Filtering.KalmanFilter import ESKF
from Code.Filtering.filter_performance_metrics import averaged_chi_squared_consistency_bands, NEES

# Problem 3: Error-State EKF in the BodyFrame

## Unicycle SE2 Dynamics
# Xdot = X xi^hat
# xi = [v; 0; omega]

## Initialize Problem Parameters
# Simulation
prob = AAE590LGM_HW4_Problem()

# Monte Carlo
N_runs = 100

def ESKF_SE2(prob):
    t_ref = prob.t_k
    X_ref = prob.simulate(np.zeros((3, prob.N_nodes)))

    def f_disc(x_k, xi_k, dt):
        x_kp1 = [x_k[0] + xi_k[0] * sp.cos(x_k[2]) * dt,
                 x_k[1] + xi_k[0] * sp.sin(x_k[2]) * dt,
                 x_k[2] + xi_k[2] * dt]
        return x_kp1

    def F_func(x_k, xi_k):
        A = np.array([[1, xi_k[2] * prob.delta_t, 0],
                      [-xi_k[2] * prob.delta_t, 1, xi_k[0] * prob.delta_t],
                      [0, 0, 1]])
        return A

    def x_update_func(x_k_km1, delta_x_k):
        x_k_k = np.zeros([3, 1])
        x_k_k[0:2] = (x_k_km1[0:2] + so2_exp(x_k_km1[2]) @ delta_x_k[0:2]).reshape((2, 1))
        x_k_k[2] = x_k_km1[2] + delta_x_k[2]

        return x_k_k

    def x_update_func_good(X_k_km1, delta_x_k):
        X_k_k = se2_mat_to_tuplevec(se2_compose(se2_tuplevec_to_mat(X_k_km1), se2_exp(delta_x_k)))
        return X_k_k

    # 2a) Prediction Step
    x_sym = sp.symbols("x:3")
    xi_sym = sp.symbols("xi:3")
    f_disc_sym = sp.Matrix(f_disc(x_sym, xi_sym, prob.delta_t))
```

```
58
59      f_disc_func = sp.lambdify((x_sym, xi_sym), f_disc_sym, "numpy")
60      def f_disc_func(X_k, xi_k):
61          X_kp1 = se2_mat_to_tuplevec(se2_compose(se2_tuplevec_to_mat(X_k), se2_exp(xi_k *
        prob.delta_t)))
62          return X_kp1
63
64      Q_k = lambda x : prob.delta_t ** 2 * prob.Q_body
65
66      # 2b) Measurement Update
67      h_GPS = lambda x : x[0:2]
68      h_GPS_noise = lambda x : prob.rng.normal(h_GPS(x), prob.sigma_GPS)
69      R_z_GPS = prob.sigma_GPS ** 2 * np.eye(2)
70      H_GPS_func = lambda x : np.hstack((so2_exp(x[2]), np.zeros((2, 1))))
71
72      # 2c) Implementation
73      xhat_0 = np.zeros((3, 1))
74      P_0 = np.diag((0.01, 0.01, 0.01))
75
76      X_true = prob.monte_carlo(1)
77      x_true = np.zeros((3, prob.N_nodes))
78      for k in range(prob.N_nodes):
79          x_true[:, k] = se2_mat_to_tuplevec(X_true[:, :, k]).reshape((3,))
80
81      xhat_k = np.zeros((3, prob.N_nodes))
82      Xhat_k = np.zeros((3, 3, prob.N_nodes))
83      Phat_k = np.zeros((3, 3, prob.N_nodes))
84      xhat_k[:, 0] = xhat_0.reshape((3,))
85      Xhat_k[:, :, 0] = se2_tuplevec_to_mat(xhat_0).reshape((3, 3))
86      Phat_k[:, :, 0] = P_0
87      nu_k = []
88      K_k = []
89      innovation = []
90      t_k_GPS = []
91      z_k_GPS = []
92      k_GPS = []
93      for k in range(prob.N_steps):
94          if np.mod(t_ref[k], 1 / prob.GPS_rate) == 0 and k > 1:
95              z_GPS = h_GPS_noise(x_true[:, k])
96              z_k_GPS.append(z_GPS)
97              t_k_GPS.append(t_ref[k])
98              k_GPS.append(k)
99
100             xhat_k[:, k + 1], Phat_k[:, :, k + 1], nu_k_i, K_k_i, innovation_i = ESKF(xhat_k
        [:, k], Phat_k[:, :, k], prob.xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k,
        H_GPS_func, R_z_GPS, x_update_func)
101
102             nu_k.append(nu_k_i)
103             K_k.append(K_k_i)
104             innovation.append(innovation_i)
105         else:
106             z_GPS = np.nan
107
108             xhat_k[:, k + 1], Phat_k[:, :, k + 1] = ESKF(xhat_k[:, k], Phat_k[:, :, k], prob
        .xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k, H_GPS_func, R_z_GPS, x_update_func)
109         Xhat_k[:, :, k + 1] = se2_tuplevec_to_mat(xhat_k[:, k + 1]).reshape((3, 3))
110
111     nu_k = np.array(nu_k)
112     K_k = np.array(K_k)
113     innovation = np.array(innovation)
114     t_k_GPS = np.array(t_k_GPS)
115     z_k_GPS = np.array(z_k_GPS)
116     k_GPS = np.array(k_GPS)
117
118     return x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS,
        k_GPS
119 x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS, k_GPS =
        ESKF_SE2(prob)
```

```
120
121 # Rotate Phat_k into world frame
122 def rotate_covariance_to_world(Xhat_k, Phat_k):
123     Rhat_k_mat = np.block([[Xhat_k[0:2, 0:2], np.zeros((2, 1))], [np.zeros((1, 2)), 1]])
124     Phat_k_world = Rhat_k_mat @ Phat_k @ Rhat_k_mat.T
125     return Phat_k_world
126
127 Phat_k_world = np.zeros_like(Phat_k)
128 for k in range(prob.N_steps):
129     Phat_k_world[:, :, k] = rotate_covariance_to_world(Xhat_k[:, :, k], Phat_k[:, :, k])
130
131 # 2d) Animation
132
133 fig, ax = plt.subplots()
134 ax.set_title("ESKF")
135 ax.set_xlabel("X Position [m]")
136 ax.set_ylabel("Y Position [m]")
137 ax.grid()
138 plt.gca().set_aspect('equal', adjustable = 'box')
139 run_plot, run_quiver, ref_plot, ref_quiver = plot_scatter_unicycles_ax(X_true[:, :, 0],
        Xhat_k[:, :, 0].reshape((3, 3, 1, 1)), True, ax)
140 cov_plot = plot_cov_ellipse_ax(Phat_k_world[0:2, 0:2, 0], xhat_k[0:2, 0], "red", "-", ax)
141 gps_plot = ax.scatter(z_k_GPS[k_GPS <= prob.N_nodes, 0], z_k_GPS[k_GPS <= prob.N_nodes, 1],
        color = "green")
142 ax.set_xlim([-10, 10])
143 ax.set_ylim([-15, 15])
144
145 def update(k):
146     ref_plot[0].set_xdata(x_true[0, :k])
147     ref_plot[0].set_ydata(x_true[1, :k])
148     run_plot[0].set_xdata(xhat_k[0, :k])
149     run_plot[0].set_ydata(xhat_k[1, :k])
150     e_data = create_cov_ellipse(Phat_k_world[0:2, 0:2, k], xhat_k[0:2, k])
151     cov_plot[0].set_xdata(e_data[0, :] + xhat_k[0, k])
152     cov_plot[0].set_ydata(e_data[1, :] + xhat_k[1, k])
153     gps_plot.set_offsets(z_k_GPS[k_GPS <= k, :])
154     return (gps_plot, cov_plot[0], ref_plot, run_plot)
155
156 ani = animation.FuncAnimation(fig=fig, func=update, frames = prob.N_nodes, interval=10)
157
158 plt.show()
159 #plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q2_ref.png")
160
161
162 # 3f) Monte Carlo Evaluation
163 nu_k_runs = np.zeros((N_runs, t_k_GPS.size))
164 eps_k_runs = np.zeros((N_runs, prob.N_nodes))
165 for i in range(N_runs):
166     x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k_runs[i, :], K_k, innovation, t_k_GPS,
        z_k_GPS, k_GPS = ESKF_SE2(prob)
167
168     Phat_k_world = np.zeros_like(Phat_k)
169     for k in range(prob.N_steps):
170         Phat_k_world[:, :, k] = rotate_covariance_to_world(Xhat_k[:, :, k], Phat_k[:, :, k])
171
172     xhat_error = x_true - xhat_k
173     xhat_error[2] = np.mod(xhat_error[2] + np.pi, 2 * np.pi) - np.pi
174
175     '''for k in range(2, prob.N_nodes - 3):
176         eps_k_runs[i, k] = NEES(xhat_error[:, k], Phat_k_world[:, :, k])'''
177     for k in range(2, prob.N_nodes - 4):
178         Xhat_error = se2_log(se2_compose(se2_inverse(Xhat_k[:, :, k]), X_true[:, :, k, 0]))
179         eps_k_runs[i, k] = NEES(Xhat_error, Phat_k[:, :, k])
180
181     print(i)
182
183 nu_k_avg = np.average(nu_k_runs, 0)
184 eps_k_avg = np.average(eps_k_runs, 0);
```

```
185
186 eps_bounds = averaged_chi_squared_consistency_bands(N_runs, 3)
187 nu_bounds = averaged_chi_squared_consistency_bands(N_runs, 2)
188
189 plt.subplot(1, 2, 1)
190 plt.plot(prob.t_k, eps_k_avg, color = "red")
191 plt.title("Time Averaged NEES")
192 plt.hlines(eps_bounds, 0, prob.T, linestyles = "--", label="95% bounds")
193 plt.grid()
194 plt.legend()
195
196 plt.subplot(1, 2, 2)
197 plt.plot(t_k_GPS, nu_k_avg, color = "red")
198 plt.title("Time Averaged NIS")
199 plt.hlines(nu_bounds, 0, prob.T, linestyles = "--", label="95% bounds")
200 plt.grid()
201 plt.legend()
202
203 plt.savefig("./HW4/Parts/Q3/AAE590LGM_HW4_Q3f_mc.png")
204
205 plt.show()
```